Teck Stack for CultivaX

🧠 STRATEGIC STACK DECISION (Finalized for CultivaX Phase 1)

I will now lock a stack that is:

- Conservative
- Clean
- Scalable to 500–600 users without rewrite
- Migration-friendly to distributed architecture
- Research-ready later

🔵 1. BACKEND (Conservative but Powerful)

✅ Python 3.11 + FastAPI

Why this is safe:

- Mature
- Async support
- High performance for moderate load
- Native ML integration
- Easy worker separation
- Clean OpenAPI auto docs

Scaling to 500–600 users?

Yes, easily.

Cloud Run can scale horizontally.
FastAPI async handles moderate concurrency well.

No rewrite required.

🔵 2. DATABASE

✅ PostgreSQL (Cloud SQL)

Why:

- ACID (important for replay)
- Row-level locking
- JSONB support
- Strong indexing
- Partition support later

Scaling plan to 600 users:

- Add read replica
- Add connection pooling
- Add proper indexes

No schema rewrite needed.

🔵 3. EVENT SYSTEM (Conservative but Migration-Ready)

✅ In-Process Partitioned Dispatcher

But with:

- Events persisted in DB
- Partition key = crop_instance_id
- Clear dispatcher abstraction interface

Important:

We design dispatcher like this:

```
class EventDispatcherInterface:
    def publish(event)
    def process(event)
```

Later migration path:

Replace implementation with Pub/Sub
No business logic change.

That’s critical.

🔵 4. ML INTEGRATION (Strong but Controlled)

Risk Model → Scikit-learn

CNN Backend → PyTorch

Edge AI → TensorFlow Lite (future-ready)

Why this hybrid is good:

- Logistic regression easy
- CNN flexible
- Edge deployable
- No external ML microservice needed

Scaling to 600 users?

Yes.
ML inference lightweight for this scale.

Heavy training can be offline batch.

🔵 5. MEDIA PROCESSING

Separate Cloud Run Worker

Backend API → queues job → worker processes

Even for 200 users:

Video worker separated prevents overload.

Scaling to 600:

Increase worker concurrency.

No rewrite needed.

🔵 6. FRONTEND

Next.js (React-based PWA)

Why conservative:

- Industry standard
- Easy hosting on Vercel or Cloud Run
- PWA ready
- Easy scaling
- Admin panel easy

No future overhaul needed.

🔵 7. DEPLOYMENT

Cloud Run Services:

Service 1 → Backend API
Service 2 → Media Worker
Service 3 → Video Worker (optional)

Cloud SQL → DB
Cloud Storage → Media
Cloud Scheduler → Cron jobs

Scaling to 600 users?

- Increase max instances
- Tune DB connections
- Add read replica

Still no major rewrite.

🔵 8. AUTHENTICATION

JWT-based auth (internal)

Later:
Can integrate Firebase Auth or OAuth
Without rewriting backend logic.

🔵 9. MONITORING

Cloud Logging + Cloud Monitoring

Add alert thresholds early.

Scaling safe.

🔥 WHY THIS IS CORRECT FOR YOUR GOAL

Because you are:

Not building Netflix
Not building Uber
Not building 100k user system

You are building:

A logic-heavy, state-sensitive, ML-bounded agricultural system.

The bottleneck is logic integrity — not request throughput.

🧠 MIGRATION READINESS PLAN (Important)

We embed these decisions NOW to avoid overhaul later:

1. No direct in-memory state reliance
2. All events persisted before processing
3. Dispatcher interface abstracted
4. ML separated logically from CTIS
5. Workers containerized
6. No synchronous heavy ML in API path
7. Feature flags implemented early

This ensures:

Moving to:

- Pub/Sub
- Redis
- Read replicas
- Horizontal scaling

Requires config change, not rewrite.

🏗 WHAT THIS STACK CAN HANDLE

Users	Status
100	Extremely safe
200	Comfortable
500	Safe with tuning
600	Manageable with DB tuning + scaling
1000+	Requires Pub/Sub + DB replica

Perfect alignment with your roadmap.